

What is backtracking?

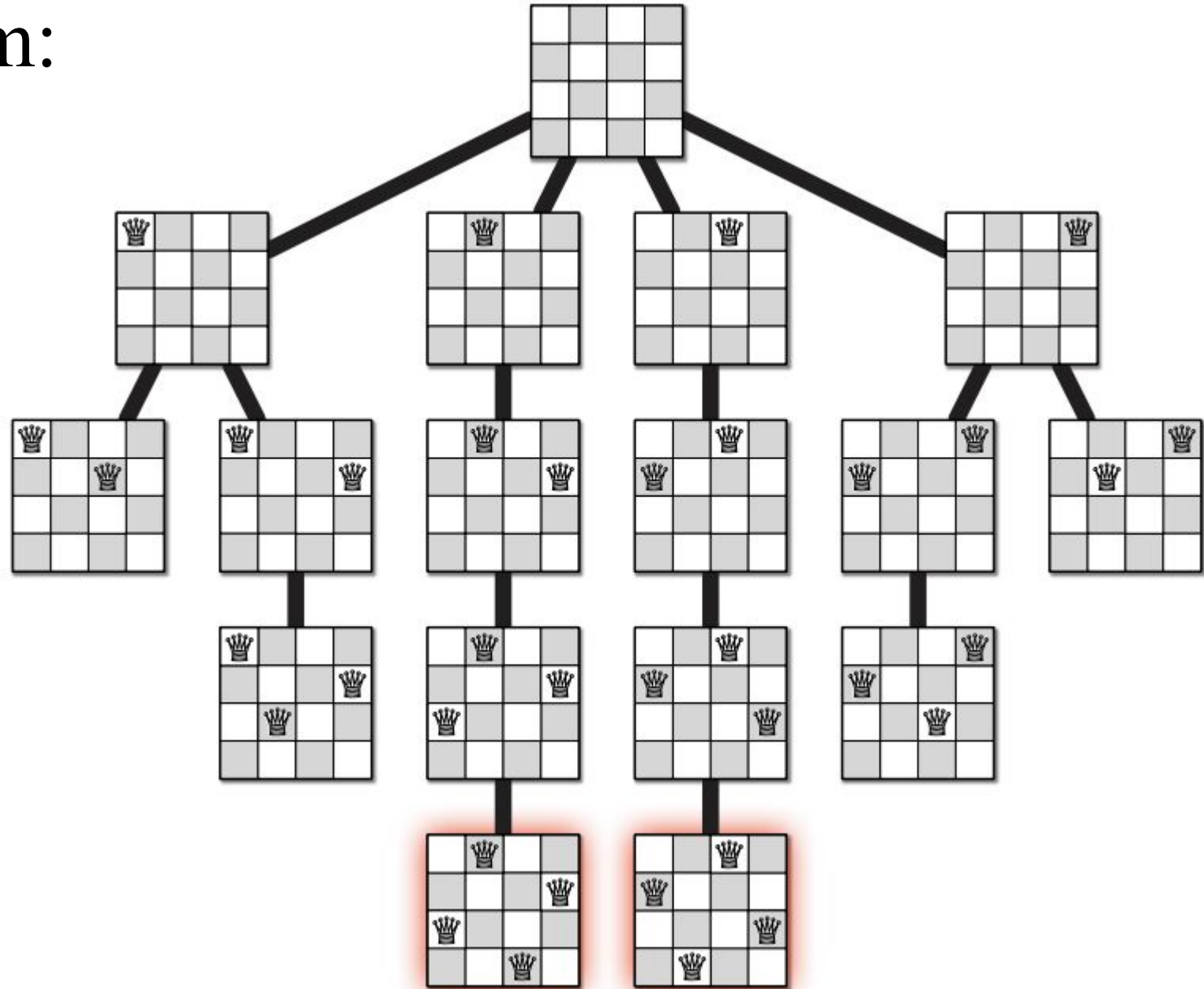
- **Backtracking** is a search technique for solving complex problems by recursively exploring combinations of possible choices to arrive at a solution.
- It is commonly used to solve search, optimization, planning and gaming problems. **Backtracking** is based on an in-depth search that explores options until a solution is found or all possibilities have been exhausted.

How Backtracking Works

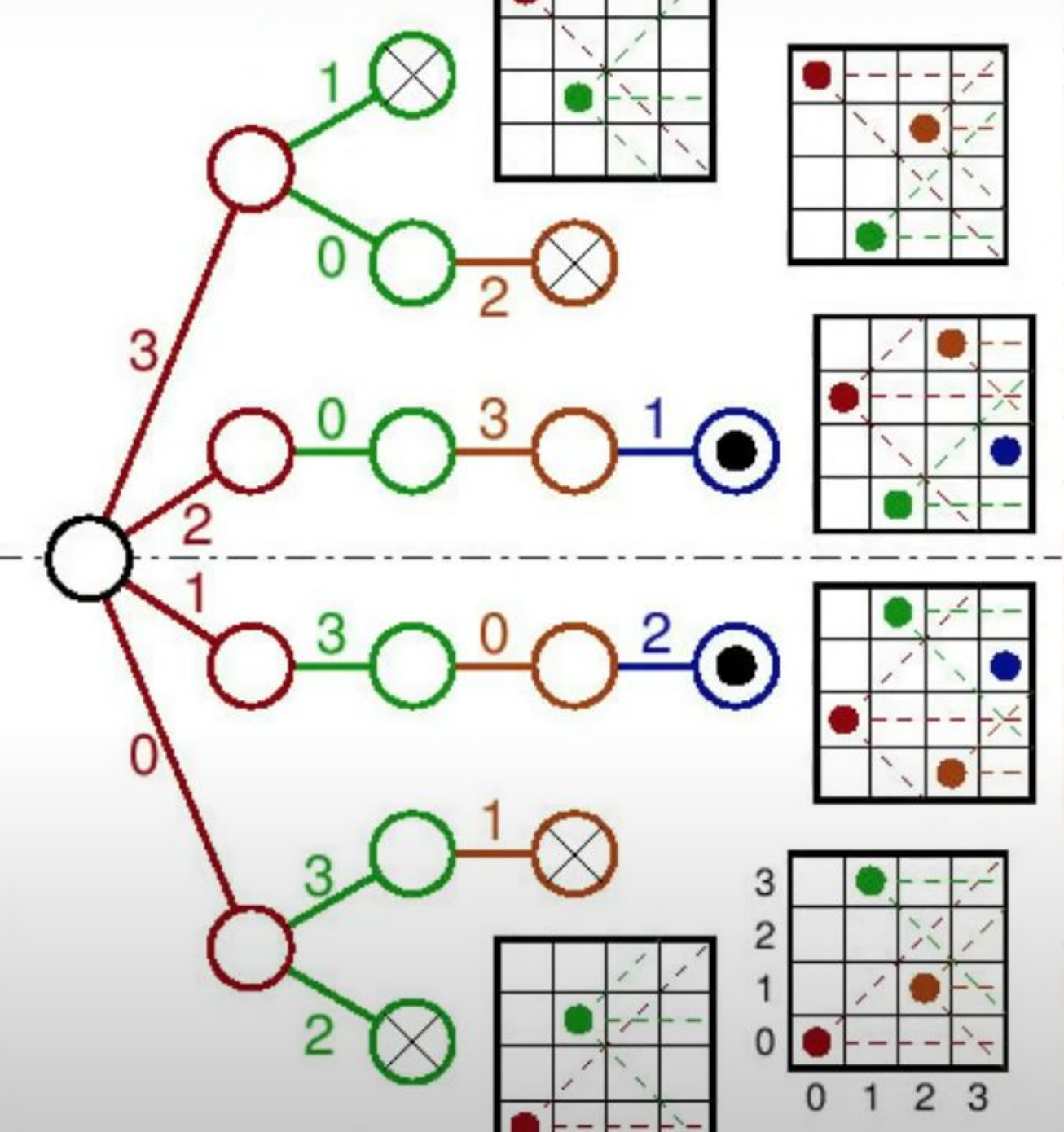
- Start with an empty solution and build it one step at a time.
- At each step, check if adding the current choice is valid:
 - If yes, move to the next step.
 - If no, backtrack by removing the current choice and trying another one.
- Repeat until all possibilities are explored or a valid solution is found.

Real-World Problem: N-Queens

- Problem: Place n queens on an $n \times n$ chessboard such that no two queens threaten each other.
- Backtracking Approach:
 1. Place queens row by row.
 2. After placing a queen, check if it leads to a valid configuration.
 3. If not, backtrack and try placing the queen in another column.

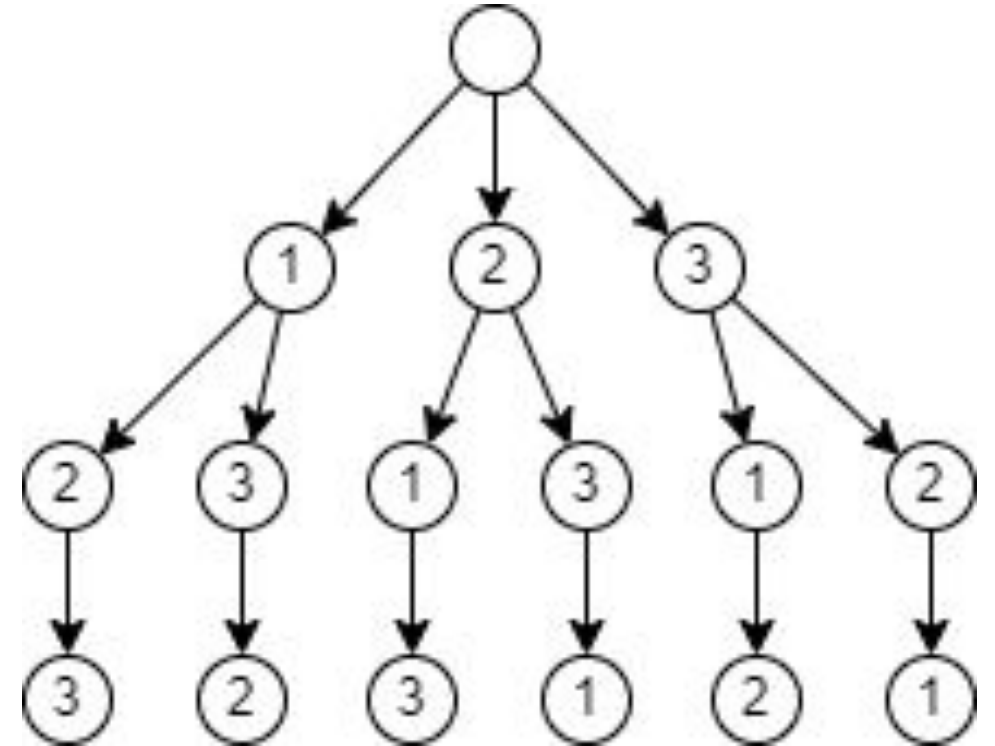


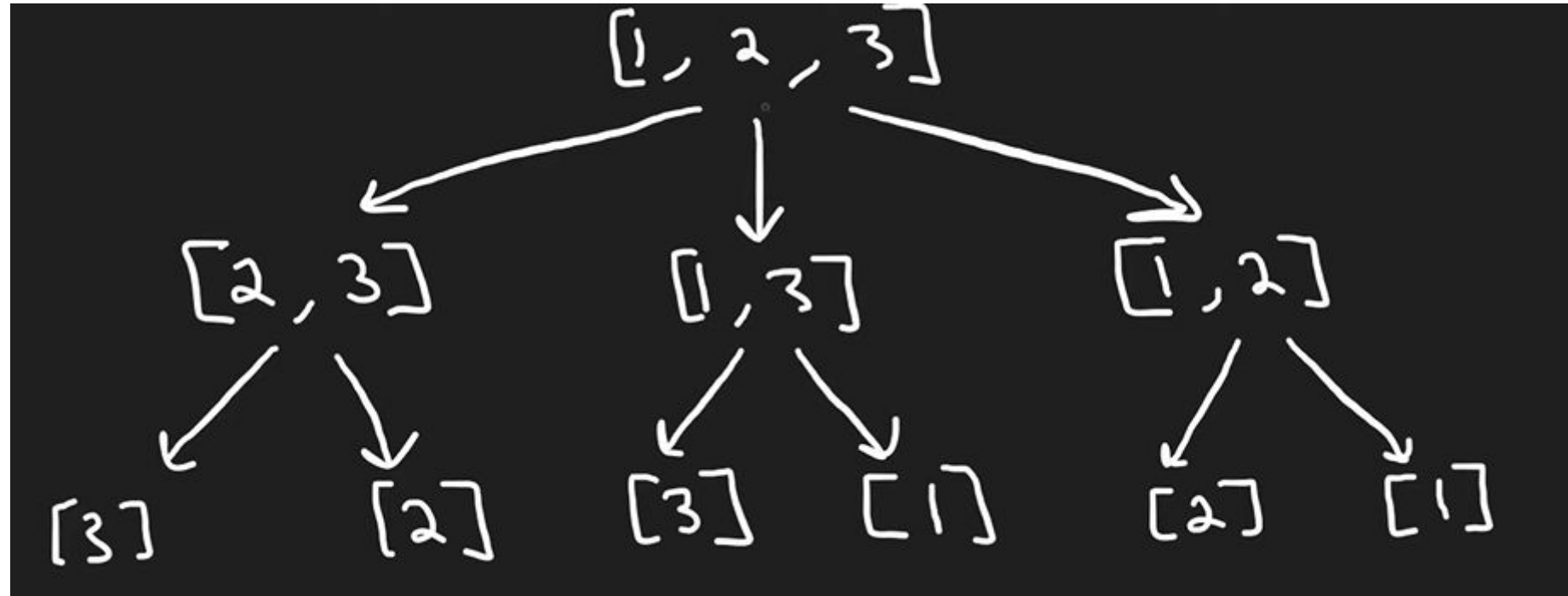
Queue



Permutation

- List of numbers = [1,2,3]
- All possible permutations
- [[1,2,3], [1,3,2],[2,1,3],[2,1,3],
[3,1,2],[3,2,1]]
- $\frac{3}{3} \times \frac{2}{2} \times \frac{1}{1} = 6$





Backtracking – Permutation of a string

A permutation also called an “arrangement number” or “order,” is a rearrangement of the elements of an ordered list S into a one-to-one correspondence with S itself. A string of length N has $N!$ permutations.

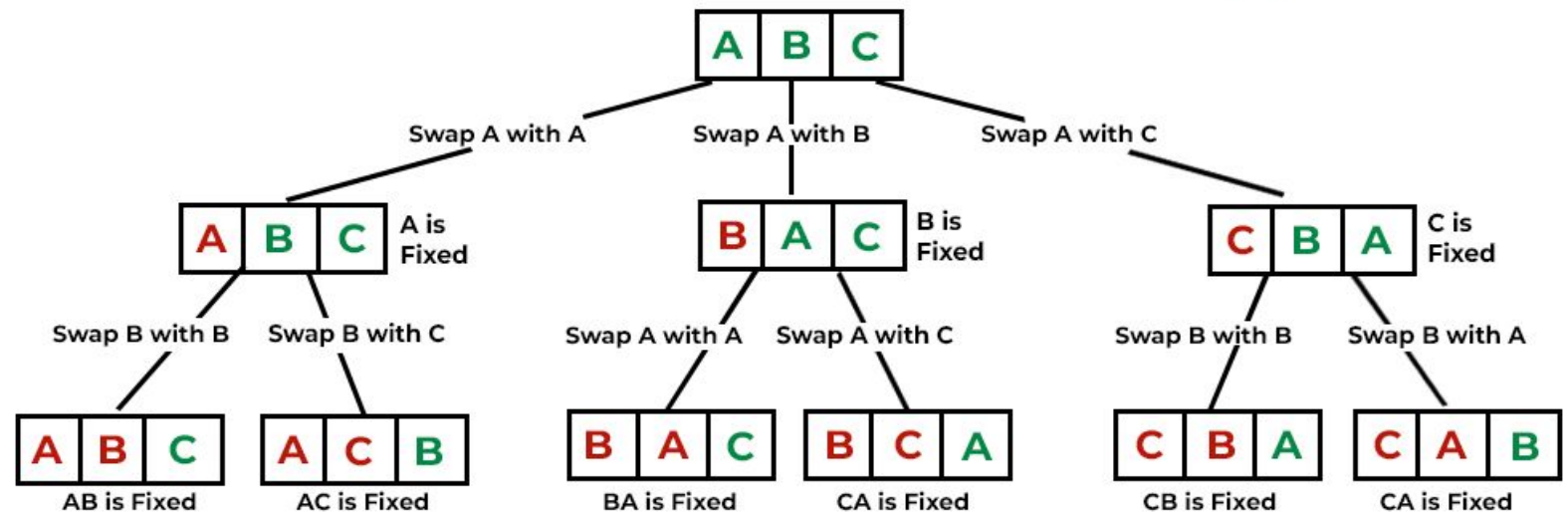
Input: $S = \text{“ABC”}$

Output: “ABC”, “ACB”, “BAC”, “BCA”, “CBA”, “CAB”

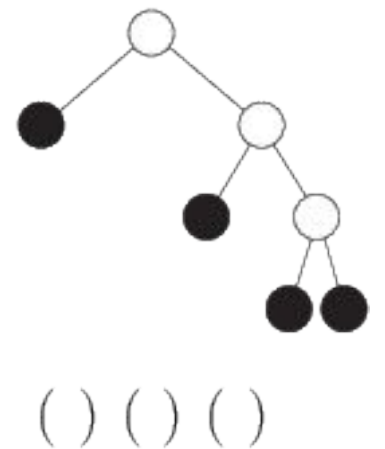
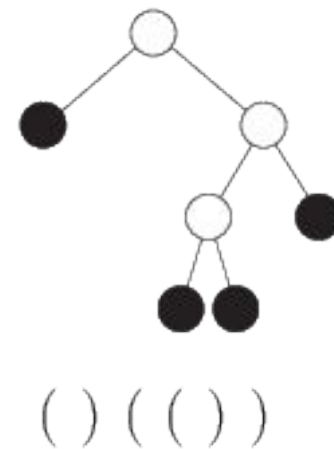
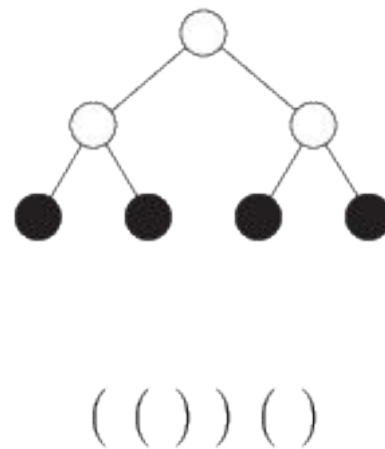
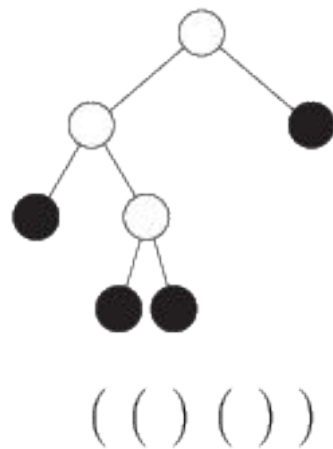
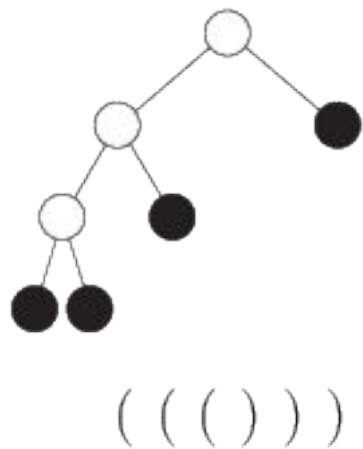
Recursion Tree for Permutations of String “ABC”



■ Fixed Characters



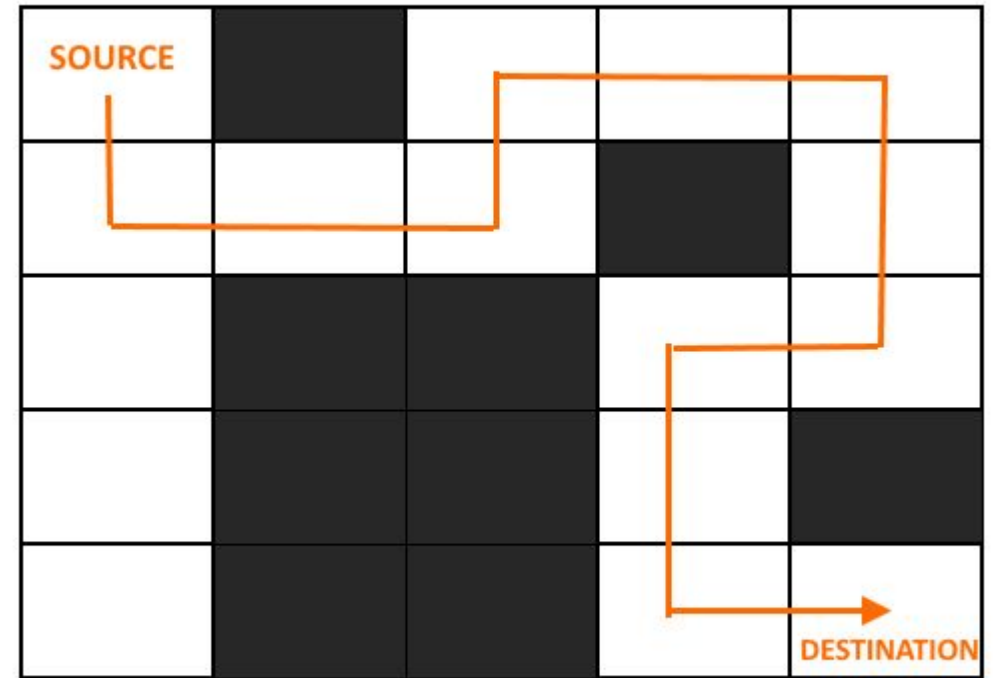
- If you have 3 pairs of parentheses, you need to generate all valid sequences with exactly 3 opening '(' and 3 closing ')' parentheses.



- Backtracking is used to explore all potential valid combinations of parentheses.
- The algorithm explores all possibilities (both adding '(' and ')'), but it prunes invalid paths (those where there are more close brackets than open brackets).
- The tree structure helps visualize the recursive decisions made at each step, and backtracking ensures that the algorithm does not explore invalid combinations further.

Backtracking - Rat In A Maze Puzzle

- To solve a maze using backtracking with marking visited paths, the algorithm needs to keep track of the cells it has already visited. This ensures that the algorithm doesn't repeatedly explore the same path, thereby improving efficiency.



Algorithm:

- If the rat reaches the destination
 - Print the solution matrix.
- Else:
 - Mark the current cell in the solution matrix as 1.
 - Move **downwards** and recursively check if this movement leads to the solution.
 - If movement in the above step doesn't lead to the solution then go towards the **right** and recursively check if this movement leads to the solution.
 - If movement in the above step doesn't lead to the solution then move **upwards** and recursively check if this movement leads to the solution.
 - If movement in the above step doesn't lead to the solution then move in the **left** direction and recursively check if this movement leads to the solution.
 - If none of the above solutions works then BACKTRACK and mark the current cell as 0.
 - In each recursive call, check the conditions like the rat is not crossing the matrix boundaries and not visiting the already visited cell.
- If the rat runs out of options, return false because it's not possible to reach to the destination